



Ingeniería de Sistemas · Curso ST1630 — Sistemas Intensivos en Datos

Pulso Medellín

Plataforma de datos híbrida para la integración de la movilidad urbana del Valle de Aburrá

Propuesta de Proyecto Final

Jean Carlo Londoño Ocampo

Moisés Vergara Garcés

Alejandro Garcés Ramírez

Medellín, 2026

1. Contexto y dominio

Medellín cuenta con uno de los ecosistemas de transporte público más complejos y diversos de América Latina: Metro, Metrocable, Tranvía, Metroplús, EnCicla y una red de buses alimentadores. Pese a esta riqueza operacional, los datos que describen esa operación están dispersos en al menos seis entidades que no comparten infraestructura técnica ni protocolos de intercambio de información.

El proyecto Pulso Medellín propone construir un sistema de datos híbrido batch y streaming que integre dichas fuentes bajo una arquitectura Lakehouse Medallion (Bronze → Silver → Gold, implementada sobre Apache Iceberg y MinIO) y un camino de tiempo real basado en Apache Kafka, Apache Flink y MongoDB. El sistema permitirá responder simultáneamente preguntas analíticas históricas y preguntas operacionales en tiempo cuasi-real, capacidad que ninguna entidad de la ciudad posee hoy de forma integrada.

La motivación no es únicamente académica. El cruce entre los datos climáticos del SIATA y los datos de movilidad —afluencia del Metro, demanda de EnCicla, volúmenes del SIMM— es un análisis que la ciudad necesita y que actualmente no existe de forma integrada en ningún portal público. Si el proyecto concluye satisfactoriamente, el dataset derivado podría publicarse en MEData como recurso abierto bajo la Ley 1712 de 2014.

2. Enunciado del problema

La pregunta central que articula el proyecto es la siguiente:

¿Cuándo llueve fuerte en la zona nororiental de Medellín, cuántos minutos de retraso se generan en el Metrocable K y cuántos usuarios migran a EnCicla o Metroplús como alternativa?

Responder esta pregunta requiere cruzar datos del SIATA, del Metro de Medellín, de EnCicla y del SIMM en un único pipeline integrado. Ese pipeline no existe hoy en la ciudad. A partir de este enunciado central se derivan las siguientes preguntas específicas, distribuidas entre el camino batch y el camino streaming del sistema:

2.1 Preguntas analíticas — camino batch

Las siguientes preguntas se responden mediante consultas históricas sobre la capa Gold del Lakehouse:

1. ¿Cuál es la correlación entre el índice de precipitación (mm/h) del SIATA y la variación de afluencia en las estaciones del Metro durante el período 2018–2024?
2. ¿En qué comunas y franjas horarias se concentra la accidentalidad vial severa según los registros de MEData, y cómo ha evolucionado esa distribución entre 2014 y 2024?
3. ¿Bajo qué umbrales de temperatura y PM2.5 cae la demanda de EnCicla más del 30 % respecto al promedio diario esperado?

4. ¿Cuáles son los corredores viales del SIMM con mayor volumen vehicular en hora pico y qué proporción de incidentes registra cada uno según MEData?

2.2 Preguntas operacionales — camino streaming

Las siguientes preguntas se responden en tiempo cuasi-real mediante consultas sobre la capa NoSQL del sistema:

5. ¿Hay estaciones EnCicla con disponibilidad menor a dos bicicletas dentro de un radio de 500 m de una estación Metro con afluencia elevada en este momento?
6. ¿El PM2.5 en alguna estación del SIATA superó el umbral de alerta ($75 \mu\text{g}/\text{m}^3$) en los últimos diez minutos?
7. ¿Cuántos vehículos por minuto están transitando por los corredores del SIMM que históricamente concentran mayor accidentalidad?
8. ¿La afluencia acumulada en alguna estación del Metro en los últimos cinco minutos supera el percentil 90 histórico para esa franja horaria?

3. Fuentes de datos y esquemas

Todas las fuentes utilizadas son públicas bajo la Ley 1712 de 2014 de Transparencia y Acceso a la Información Pública. Los conjuntos de datos han sido descargados y revisados; los problemas de calidad que se describen a continuación son reales y provienen de las fuentes originales.

3.1 Incidentes viales — Secretaría de Movilidad (MEData)

Atributo	Descripción
Portal / API	medata.gov.co · Socrata API
Formato	CSV / JSON
Volumen	Más de 150 000 registros (2014–2024)
Frecuencia	Actualización mensual por parte de la entidad. Tipo: dato histórico (camino batch).
Campos clave	fecha_accidente, hora, clase_accidente, gravedad, latitud, longitud, barrio, comuna, num_heridos, num_fallecidos
Problemas de calidad	Coordenadas invertidas en aproximadamente el 3 % de registros previos a 2017; el campo 'barrio' fue renombrado a 'barrio_accidente' en 2017, generando incompatibilidad de esquema; duplicados por doble reporte policial.
Uso en el pipeline	Bronze: ingesta cruda con partición por fecha de ingesta. Silver: deduplicación y join espacial con GeoMedellín. Gold: accidentalidad por comuna normalizada y mapa de puntos negros de siniestralidad.

3.2 Afluencia Metro + GTFS — Metro de Medellín

Atributo	Descripción
Portal / API	datosabiertos-metrodemedellin.opendata.arcgis.com
Formato	CSV (afluencia), GeoJSON (estaciones), GTFS (rutas y horarios)
Volumen	Aproximadamente 2 millones de registros de afluencia mensual; 27 estaciones Metro y 10 Metrocable
Frecuencia	Histórico mensual (batch). El productor Kafka simulará validaciones de torniquete cada 30 segundos (streaming).
Campos clave	estacion_id, nombre_estacion, fecha, hora, pasajeros_entrada, pasajeros_salida, linea
Problemas de calidad	El GTFS presenta stop_times faltantes para estaciones auxiliares; la afluencia está agregada sin desglose por torniquete.
Uso en el pipeline	Bronze: CSV por lotes mensuales. Silver: join temporal con registros SIATA por hora. Gold: correlación PM2.5 vs. afluencia. Streaming: validaciones simuladas en tiempo real para responder la pregunta S-4.

3.3 EnCicla — préstamos y disponibilidad (Área Metropolitana)

Atributo	Descripción
Portal / API	datosabiertos.metropol.gov.co
Formato	CSV (préstamos históricos), API JSON (disponibilidad en tiempo real)
Volumen	128 000 usuarios registrados; 103 estaciones; aproximadamente 1 millón de préstamos por año
Frecuencia	Histórico mensual (batch). Disponibilidad actualizada cada 5 minutos via API (streaming).
Campos clave	id_prestamo, estacion_salida_id, estacion_llegada_id, fecha_hora_salida, fecha_hora_llegada, duracion_min, disponibilidad_bicicletas
Problemas de calidad	Duraciones negativas por relojes desincronizados; inconsistencias en el campo id_estacion entre el CSV histórico y la API de disponibilidad.
Uso en el pipeline	Batch: análisis de demanda cruzada con precipitación (pregunta B-3). Streaming: disponibilidad por estación en tiempo real para la alerta de rebalanceo (pregunta S-1). Requiere anonimización de datos de usuarios bajo la Ley 1581 de 2012.

3.4 SIATA — meteorología y calidad del aire

Atributo	Descripción
Portal / API	siata.gov.co (tiempo real) + Kaggle (histórico 2017–2023)
Formato	JSON (tiempo real), CSV (histórico)

Volumen	Aproximadamente 40 estaciones; una lectura cada 10 minutos; histórico con más de 3 millones de registros
Frecuencia	Cada 10 minutos (streaming principal del sistema). Histórico Kaggle disponible por año (batch).
Campos clave	estacion_id, latitud, longitud, timestamp, pm25, pm10, temperatura_c, humedad_pct, precipitacion_mm, viento_kmh
Problemas de calidad	Brechas por mantenimiento de sensores; valores de PM2.5 igual a -999 usados como nulos; timestamps sin zona horaria explícita.
Uso en el pipeline	Streaming principal: JSON cada 10 minutos → Kafka → Flink (ventana tumbling de 10 minutos) → MongoDB (alerta S-2). Batch: histórico Kaggle → Bronze → Silver, join temporal con afluencia del Metro (pregunta B-1).

3.5 GeoMedellín — comunas, barrios y malla vial

Atributo	Descripción
Portal / API	geomedellin.gov.co / opendata
Formato	GeoJSON, Shapefile
Volumen	16 comunas, 249 barrios, aproximadamente 400 km de ciclorrutas
Frecuencia	Actualización anual. Fuente dimensional estática para el camino batch.
Campos clave	comuna_id, nombre_comuna, barrio_id, nombre_barrio, geometry (Polygon / MultiPolygon)
Problemas de calidad	Proyección CRS mixta entre archivos del mismo año (EPSG:4326 y EPSG:3116); requiere reproyección antes del join espacial.
Uso en el pipeline	Silver: join espacial ST_Within para asignar la comuna y el barrio correspondientes a cada incidente vial y a cada estación EnCicla.

3.6 SIMM — aforos vehiculares por corredor

Atributo	Descripción
Portal / API	medata.gov.co (tableros SIMM)
Formato	CSV descargable por corredor
Volumen	Aproximadamente 30 corredores; aforos hora a hora desde 2019
Frecuencia	Histórico semanal (batch). El productor Kafka simulará conteos cada 60 segundos (streaming).
Campos clave	corredor_id, nombre_corredor, fecha, hora, volumen_vehiculos, composicion (autos, motos, buses, camiones)
Problemas de calidad	Brechas de datos durante la cuarentena de 2020; ausencia de coordenadas geográficas, lo que requiere un join manual con la malla vial de GeoMedellín.

Uso en el pipeline	Batch: análisis de corredores de alto riesgo compuesto (volumen + accidentalidad, pregunta B-4). Streaming: alerta de corredor activo en zona de alta siniestralidad (pregunta S-3).
--------------------	--

4. Arquitectura del sistema

El sistema integra dos caminos de procesamiento complementarios sobre el mismo dominio. El camino batch responde preguntas históricas y analíticas; el camino streaming responde preguntas operacionales en tiempo cuasi-real. Ambos caminos son necesarios y no sustituibles entre sí: las preguntas B-1 a B-4 requieren el análisis de varios años de datos, mientras que las preguntas S-1 a S-4 exigen respuestas en ventanas de segundos a minutos.

Toda la infraestructura se despliega mediante Docker Compose para garantizar la reproducibilidad local completa del pipeline sin intervención manual más allá de un único comando de arranque.

4.1 Camino batch — Lakehouse Medallion con Apache Iceberg

El camino batch sigue el patrón Medallion de tres capas:

- **Capa Bronze.** Zona de aterrizaje sin transformaciones. Spark (PySpark) realiza al menos dos lotes de append por fuente, garantizando que el historial de ingestiones se preserve. Cada registro incorpora metadatos de auditoría: `timestamp_ingesta`, `nombre_archivo` y `fuentes_id`. La partición se realiza por fuente y fecha de ingesta. El formato de almacenamiento es Apache Iceberg sobre MinIO.
- **Capa Silver.** Zona de limpieza y enriquecimiento. Se realizan deduplicación por clave compuesta, corrección de coordenadas invertidas, casting de tipos incorrectos e imputación documentada de nulos. Los joins críticos son: (1) join espacial con GeoMedellín para asignar comuna y barrio a cada incidente vial y a cada estación; (2) join temporal por hora entre lecturas SIATA y afluencia del Metro. La salida se almacena en Parquet con compresión Snappy. En esta capa se activa la funcionalidad de Schema Evolution de Iceberg al agregar las columnas calculadas.
- **Capa Gold.** Zona de métricas de negocio. Contiene las agregaciones que responden directamente las preguntas B-1 a B-4: correlación PM2.5 vs. afluencia, accidentalidad por comuna normalizada, elasticidad de demanda de EnCicla y ranking de corredores por riesgo compuesto. Las tablas Gold se gestionan con las funcionalidades ACID y Time Travel de Iceberg, con snapshots auditables. El motor de consulta principal es Spark SQL; Trino se incorporará como tercer motor para demostrar la interoperabilidad multi-motor que distingue a Iceberg de los formatos propietarios.

4.2 Camino streaming — Kafka, Flink y MongoDB

El camino streaming procesa eventos de dominio en tiempo cuasi-real mediante los siguientes componentes:

- **Productores Kafka.** Scripts en Python que publican eventos con esquema JSON coherente con el dominio: validaciones de torniquete del Metro (`estacion_id`, `timestamp`, `tipo_evento`), lecturas del SIATA cada 10 minutos, disponibilidad de bicicletas EnCicla cada 5 minutos y conteos de aforos SIMM cada 60 segundos. Cada tópico contará con al menos dos particiones y retención configurada explícitamente (mínimo 7 días para replay de eventos).
- **Apache Flink.** Jobs de procesamiento con estado que consumen de Kafka, deserializan JSON y aplican transformaciones con ventanas temporales: ventana tumbling de 10 minutos para agregar PM2.5 por zona (pregunta S-2); ventana tumbling de 5 minutos para acumular validaciones del Metro por estación (pregunta S-4); ventana sliding de 1 minuto con paso de 30 segundos para monitoreo de disponibilidad EnCicla (pregunta S-1). El checkpointing estará habilitado con semántica at-least-once; se evaluará exactly-once con offsets transaccionales de Kafka si el entorno lo permite.
- **MongoDB (NoSQL).** Base de datos documental elegida por su esquema flexible — apropiado para eventos heterogéneos con estructuras distintas según la fuente— y por su acceso de baja latencia mediante índices compuestos por (`zona`, `timestamp`) y (`estacion_id`). Flink persiste los resultados de las ventanas en colecciones MongoDB: `alertas_aire`, `disponibilidad_encicla`, `afluencia_metro_rt` y `aforos_corredor`. La justificación frente a una base de datos SQL radica en la escalabilidad horizontal para ingestión continua sin bloqueos de esquema y en la capacidad de consulta por clave primaria compuesta sin escaneo completo.

4.3 Integración entre caminos

Los dos caminos no son independientes. La pregunta que ilustra su integración es la siguiente: ¿el retraso histórico del Metrocable K en días de lluvia —establecido como umbral en la capa Gold (pregunta B-1)— se está materializando hoy en tiempo real? Para responderla, el job Flink de afluencia del Metro (pregunta S-4) consulta el percentil histórico almacenado en Gold y, si la afluencia actual cae por debajo de ese umbral para la franja horaria y nivel de precipitación actuales, emite una alerta en MongoDB. Esto convierte al histórico analítico en un parámetro operacional del sistema de tiempo real.

4.4 Cuándo consultar MongoDB vs Gold

Criterio	MongoDB — consultas operacionales	Gold Iceberg — consultas analíticas
Latencia	Menor a 10 ms (lectura por clave primaria)	Segundos a minutos (scan y agregación)
Cardinalidad	Baja: últimas N alertas o una estación concreta	Alta: millones de filas históricas con agregaciones
Actualizaciones	Escrituras continuas desde Flink sin bloqueo de esquema	Micro-batch o batch nocturno; transacciones ACID de Iceberg
Ejemplo	¿Cuántas bicicletas hay en la estación El Poblado ahora? → consulta por estacion_id (S-1)	¿Cuál es la correlación histórica PM2.5–afluencia entre 2018 y 2024? → Spark SQL sobre Gold (B-1)

5. Justificación del stack tecnológico

Cada componente del stack está seleccionado en función de una necesidad concreta del dominio, no por defecto.

Componente	Tecnología	Justificación en el contexto del proyecto
Infraestructura	Docker Compose	Permite que cualquier evaluador reproduzca el pipeline completo con un único comando. Cada servicio —Kafka, Flink, Spark, MinIO, MongoDB y el REST Catalog de Iceberg— se declara como un servicio con health check verificable.
Mensajería	Apache Kafka	Desacopla los productores de eventos (SIATA, Metro, EnCicla, SIMM) de los consumidores (Flink). Permite gestionar los tres SLAs distintos del proyecto —alertas en menos de 30 segundos, dashboards cada 15 minutos, reportes mensuales— mediante tópicos independientes con retención diferenciada.
Streaming	Apache Flink	Único motor del curso con soporte nativo de ventanas con estado (tumbling, sliding, session), checkpointing tolerante a fallos y semántica de entrega configurable. Necesario para las preguntas S-1 a S-4, donde las respuestas deben estar disponibles en ventanas de segundos a minutos.
Procesamiento batch	Apache Spark (PySpark)	Motor unificado para ETL masivo (más de 150 000 incidentes, millones de registros de afluencia), joins geoespaciales mediante UDFs, y procesamiento de legado con MapReduce para los archivos CSV previos a 2017 con esquema incompatible.
Formato de tabla	Apache Iceberg + REST Catalog	Iceberg aporta ACID, Time Travel y Schema Evolution que Parquet plano no ofrece. El REST Catalog permite que Spark, Trino y notebooks de Python lean las mismas tablas Gold sin duplicación de datos. La interoperabilidad multi-motor es la ventaja diferencial frente a Delta Lake, que presenta dependencias más estrechas con el ecosistema Databricks.

Almacenamiento de objetos	MinIO (local) / S3 o GCS (nube)	MinIO provee en Docker una interfaz compatible con la API de S3, lo que garantiza paridad local-nube sin cambios en el código. Es el almacén subyacente del Lakehouse y del checkpoint storage de Flink.
Base de datos NoSQL	MongoDB	La heterogeneidad de los eventos de las cuatro fuentes de streaming (estructuras JSON distintas) favorece un esquema documental flexible. Los índices compuestos por (zona, timestamp) y (estacion_id) responden las preguntas operacionales S-1 a S-4 en tiempo de acceso por clave, sin escaneos completos.
Procesamiento legado	Hadoop MapReduce	El corpus de incidentes viales incluye archivos CSV con dos esquemas distintos (anterior y posterior a 2017) y formatos sin encabezado en algunos años. MapReduce permite procesar este legado de forma distribuida antes de ingestarlo en Bronze, cubriendo el eje del curso.

6. Decisiones de arquitectura pendientes

6.1 Lambda vs. Kappa

El proyecto presenta tres SLAs simultáneos: alertas en menos de 30 segundos (SIMM), dashboards actualizados cada 15 minutos (Secretaría de Movilidad) y reportes mensuales de EnCicla (Área Metropolitana). La arquitectura Lambda mantiene los dos caminos separados con mayor madurez operacional y facilita el re-procesamiento histórico masivo sin afectar el SLA de alertas; la arquitectura Kappa unifica el procesamiento en streaming pero complejiza el análisis de 10 años de incidentes viales. La hipótesis de partida es Lambda; el documento de decisión evaluará si las ventajas de Kappa justifican la migración.

6.2 Delta Lake vs. Apache Iceberg (o esquema híbrido)

La capa Gold tiene cuatro consumidores con motores distintos: Databricks SQL, Snowflake, Amazon Athena y Python (Colab/Jupyter). Delta Lake presenta menor interoperabilidad fuera del ecosistema Databricks; Iceberg con REST Catalog es leído de forma nativa por todos los motores mencionados. La hipótesis de partida es Iceberg para toda la capa Gold; el benchmark del módulo de formatos evaluará si algún consumidor específico requiere una capa Delta paralela.

6.3 AWS vs. GCP como proveedor cloud principal

AWS ofrece mayor compatibilidad con el stack del curso (MinIO $\approx$ S3 API, Glue, Athena) y mayor disponibilidad de créditos académicos. GCP ofrece mejor integración nativa con BigQuery y menores costos de egress hacia ese motor. La hipótesis de partida es AWS; la decisión final dependerá de la disponibilidad de créditos del grupo al momento de la implementación.

7. Gobernanza y privacidad

EnCicla expone datos de 128 000 usuarios registrados con número de cédula de ciudadanía. El sistema aplicará las siguientes medidas de protección de datos antes de que cualquier información personal ingrese al Lakehouse, en cumplimiento de la Ley 1581 de 2012:

- **Pseudonimización** por HMAC-SHA256 del campo `id_usuario` en el momento de la ingesta, antes de la escritura en Bronze. La clave de reversión se almacena fuera del Lakehouse y no es accesible desde los notebooks de análisis.
- **Supresión en Silver de los campos directamente identificables** (nombre, cédula, correo electrónico). Solo se conservan el identificador pseudónimo, el id de estación y el timestamp.
- **K-anonimización** con k mayor o igual a 5 aplicada en las tablas de viajes individuales de la capa Gold.

8. Módulos de entrega

El proyecto se organiza en siete módulos, cada uno diseñado para cubrir al menos un eje temático del curso:

#	Módulo	Cobertura del curso
01	Arqueología de datos: procesamiento con MapReduce de los archivos CSV legacy de incidentes viales (dos esquemas distintos, formato sin encabezado en algunos años).	Hadoop MapReduce — contenidos T2
02	Documento de decisión Lambda vs. Kappa: análisis de los tres SLAs simultáneos del proyecto y justificación de la arquitectura seleccionada.	Arquitectura de sistemas de datos
03	Implementación completa del Medallion Bronze → Silver → Gold sobre Apache Iceberg y MinIO en Docker Compose, con datos reales.	Lakehouse, Apache Iceberg — contenidos T5/T6
04	Benchmark de formatos: comparación CSV vs. Parquet vs. ZSTD con métricas de tamaño, velocidad de escritura y lectura, y demostración de partition pruning.	Formatos de almacenamiento — contenidos T3/T4
05	Estrategia multi-consumidor: evaluación de Delta Lake vs. Iceberg para los cuatro consumidores del sistema (Databricks, Snowflake, Athena, Colab).	Interoperabilidad Iceberg — contenidos T6
06	Modelos con Spark: (a) predicción de fatalidad en incidentes viales con MLlib; (b) cálculo de rutas de menor tiempo en la red del Metro con GraphX.	Spark MLlib y GraphX — contenidos T4

07	Cloud, gobernanza y privacidad: Architecture Decision Record de proveedor cloud, controles de acceso por capa, anonimización de datos de EnCicla y marco de cumplimiento con la Ley 1581.	Cloud y gobernanza de datos
Bonus	Trino como tercer motor SQL sobre Gold; Makefile que orqueste el pipeline completo en un solo comando; Jupyter Notebook de análisis exploratorio sobre la capa Gold.	Interop multi-motor (+2 pt); orquestación (+1 pt); análisis exploratorio (+1 pt)

Cada módulo será entregado con su código correspondiente, pruebas de ejecución y documentación técnica en el repositorio del proyecto.